FECAP
Ciência da Computação — 6º semestre — 2026

Modelagem formal do sistema

Smart Tennis Arena — alvo interativo TOBIAS

Máquina de estados revista, regras, pré e pós-condições, modelo computacional e decidibilidade

Teoria da Computação e Linguagens Formais
Entrega 2

Integrantes
Gabriel Carvalho Mota — 24025708
Guilherme de Lima Siqueira — 21010649
Rodrigo Luiz Menezes dos Reis — 24036593
Vitória Leticia Maciel da Silva — 24026593

São Paulo — 2026

Sumário

Contexto do projeto **3**

1. Atualização da máquina de estados **3**

1.1 Tabela de transições **3**

1.2 Sub-máquina da calibração **4**

2. Descrição dos eventos **5**

Dois eventos mudaram de dono **5**

O que nós deixamos de fora do alfabeto **5**

3. Regras de funcionamento **5**

4. Pré e pós-condições **6**

5. Modelo computacional **7**

Por que o autômato finito basta **7**

Onde o sistema sai da classe regular **8**

A conclusão **8**

6. Análise de decidibilidade **8**

Por que elas são determinísticas **9**

Um limite que vale deixar registrado **9**

7. Divergências em relação à Entrega 1 ..... **9**

A garantia da Entrega 1 continua valendo **10**

Uma simplificação que nós mantivemos **10**

Contexto do projeto

O nosso projeto é um alvo interativo em forma de boneco joão-bobo, o TOBIAS. O praticante acerta o alvo com a bola de tênis, um sensor de aceleração dentro do boneco detecta a batida, o próprio dispositivo reage na hora com som, e manda o evento por Bluetooth para o aplicativo, que conta os pontos e cuida dos dados.

Na Entrega 1 nós modelamos o ciclo de vida de uma sessão de treino, do pareamento até o encerramento. Agora que o sistema está construído, este documento revisa aquela modelagem para refletir o que de fato ficou de pé. O recorte continua o mesmo, o que mudou foi o sistema, e cada diferença está justificada.

1. Atualização da máquina de estados

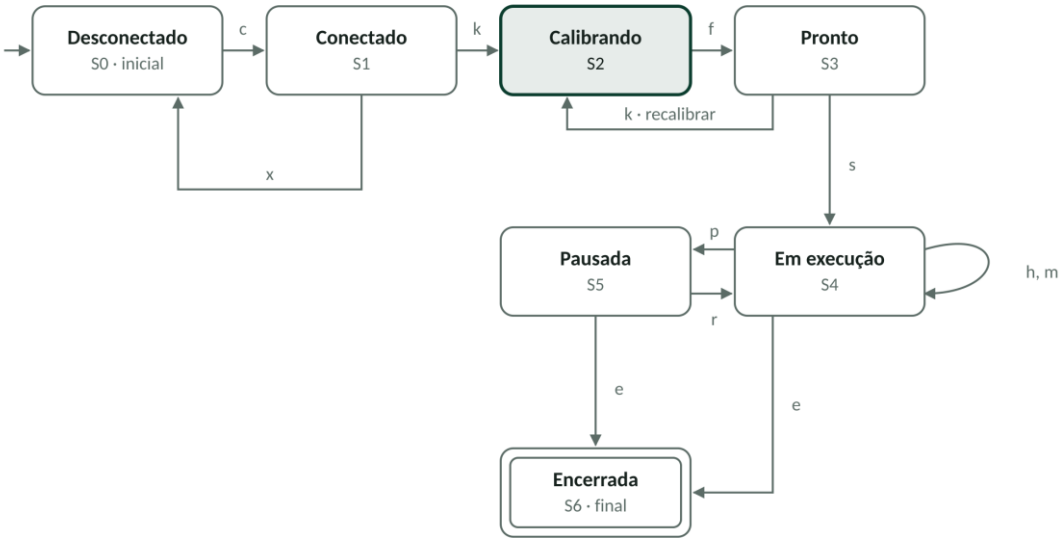
O caminho principal é o mesmo da Entrega 1: conectar, calibrar, iniciar, executar e encerrar. O que mudou foram três coisas que nós só descobrimos depois de construir o sistema.

A primeira é o evento **x**, a queda da conexão. Na Entrega 1 a gente tratou o pareamento como uma via de mão única: uma vez conectado, nunca mais se voltava a S0. Na prática o Bluetooth cai, e deixar isso de fora tornava o modelo incompleto.

A segunda é que **k** passou a sair também de S3. No aplicativo existe a opção de recalibrar um alvo que já está pronto, pelo perfil, e essa seta não existia antes.

A terceira está dentro de S2: o que era um evento instantâneo virou uma contagem de cinco batidas, e por isso fizemos um diagrama só para esse processo. Por isso também apareceu o evento **f**, que fecha a calibração. Na Entrega 1, k abria e fechava sozinho.

O estado de erro **SE** continua fora do desenho, pela mesma convenção que usamos na Entrega 1: todo par de estado e símbolo que não tem seta leva para ele, o comando é descartado e registrado no log, e a cadeia nunca é aceita.



Máquina de estados da sessão de treino, revista após a implementação. Nenhuma batida chega a ser registrada antes da calibração: toda seta que chega em S4 vem de S3.

1.1 Tabela de transições

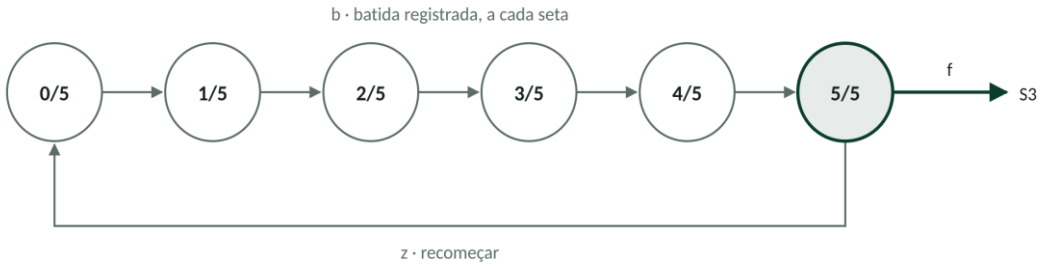
Estado	Evento	Próximo	Ação
S0	c	S1	Pareia e recebe o estado do dispositivo

Estado	Evento	Próximo	Ação
S1	k	S2	Envia CALIBRAR e abre a contagem de batidas
S1	x	S0	Conexão perdida. O alvo volta a se anunciar
S2	f	S3	Calcula a mediana e grava a referência
S2	x	S0	Calibração abortada. A referência anterior permanece
S3	s	S4	Envia TREINO:n e abre a sessão
S3	k	S2	Recalibração a partir do perfil
S3	x	S0	Conexão perdida
S4	h	S4	Registra o acerto e incrementa a sequência
S4	m	S4	Registra o erro e zera a sequência
S4	p	S5	Para de contar, preservando o contexto
S4	e	S6	Fecha a sessão e monta o resumo
S5	r	S4	Retoma a contagem do ponto em que parou
S5	e	S6	Fecha a sessão que estava pausada
S6	qualquer	SE	A sessão acabou. Outra exige uma nova cadeia
qualquer	sem seta	SE	Descarta o comando e registra a tentativa

Como para cada par de estado e símbolo existe um único destino, a máquina permanece determinística.

1.2 Sub-máquina da calibração

O estado S2 esconde um contador. Cada batida registrada avança uma posição, e só a última, com as cinco batidas acumuladas, habilita o `f`. O evento `z`, que é o botão “Recomeçar” da tela, zera o contador de qualquer posição.



Refinamento do estado S2: contador limitado, seis posições. A calibração só conclui com exatamente cinco batidas.

Esse refinamento serve para mais do que descrever a tela. Ele mostra que a regra R3 não é uma promessa do código: é uma propriedade do desenho. Não existe caminho saindo de 0/5, 1/5, 2/5, 3/5 ou 4/5 até S3, porque a única seta rotulada `f` sai da posição 5/5.

E é o exemplo mais claro do argumento que aparece na seção 5. Um contador com teto conhecido cabe num autômato finito, bastando um estado por valor: cinco batidas custam seis estados. Se a calibração pedisse um número de batidas definido só na hora, sem limite, nenhuma quantidade fixa de estados daria conta.

2. Descrição dos eventos

São dez eventos, dois a mais que na Entrega 1. Alguns são comandos que o usuário dispara pelo aplicativo, outros nascem do próprio dispositivo ou de um temporizador. Para a máquina de estados, tanto faz: os dois tipos são apenas símbolos de entrada.

```
Σ = { c, x, k, f, s, h, m, p, r, e }
```

Sím.	Evento	Quem gera	O que acontece
c	conectar	Aplicativo	Pareia com o alvo por Bluetooth e recebe nome, bateria e estado
x	desconectar	Dispositivo	A conexão cai ou não se estabelece, e o alvo volta a se anunciar
k	iniciar calibração	Aplicativo	Envia CALIBRAR e abre a contagem das cinco batidas
f	concluir calibração	Aplicativo	Tira a mediana das cinco batidas e grava a referência
s	iniciar sessão	Aplicativo	Envia TREINO:n com os parâmetros da versão do treino
h	acerto	Dispositivo (sensor)	Detecta o impacto, toca o áudio e registra o lance
m	sem impacto	Aplicativo (temporizador)	A janela do lance fechou sem batida: registra o erro e zera a sequência
p	pausar	Aplicativo	Para de contar, guardando a pontuação e a sequência
r	retomar	Aplicativo	Volta a contar de onde parou
e	encerrar	Aplicativo, ou o próprio treino ao acabarem os lances	Fecha a sessão e monta o resumo

Dois eventos mudaram de dono

Na Entrega 1 nós colocamos o `m` como sendo gerado pelo dispositivo, pelo temporizador do sinal de chamada. Isso não se sustentou: o sensor inercial só consegue avisar quando algo acontece, e **erro é a ausência de impacto**. Quem sabe que uma janela de lance passou em branco é o aplicativo, porque é ele que conhece o intervalo configurado no treino. Então o `m` passou a ser dele.

O `f` simplesmente não existia. Antes, `k` abria e fechava a calibração num símbolo só. Agora `k` abre a contagem e `f` a fecha, porque entre os dois existem cinco batidas para registrar.

O que nós deixamos de fora do alfabeto

A classificação do impacto em fraco, médio, forte ou KO viaja junto com o `h`, mas ficou de fora. É a mesma decisão que tomamos na Entrega 1 para os valores numéricos: a faixa de ângulo não altera nenhuma transição, então colocá-la no alfabeto multiplicaria os símbolos sem mudar o comportamento de nada.

3. Regras de funcionamento

Estas são as regras que governam o comportamento do sistema. Escrevi cada uma de um jeito que dá para conferir no autômato: ou ela corresponde a uma seta que existe, ou a uma seta que nós deliberadamente não desenhamos.

R1. Se o alvo não estiver pareado, nenhum comando de treino é aceito. No diagrama, S0 não tem seta saindo por s.

R2. Se o alvo estiver pareado mas sem referência de calibração, a sessão não começa. Na tela que lista as sessões de treino, o botão “Iniciar sessão” só habilita com o alvo marcado como Pronto.



R3. A calibração só conclui com exatamente cinco batidas. Com menos que isso, o botão “Concluir calibração” fica desabilitado e a única saída é recomeçar.

R4. Se a sessão estiver pausada, nenhum impacto é registrado. S5 não tem seta por h nem por m.

R5. Só dá para retomar o que foi pausado. Como não existe seta por r saindo de S4, é impossível retomar sem ter pausado antes.

R6. Sessão encerrada não volta atrás. S6 é absorvente: qualquer símbolo que chegue ali cai no estado de erro, e para treinar de novo é preciso uma cadeia nova.

R7. A pontuação só aparece depois de encerrar. Durante a sessão, a tela da sessão em andamento mostra apenas o tempo, porque os lances ficam guardados no dispositivo até o fim.

R8. Se a conexão cair, o sistema volta a S0 de onde estiver. A referência de calibração não se perde, porque fica gravada na memória não volátil do ESP32.

R9. Uma sessão encerrada é sincronizada pelo menos uma vez, e reenviar não duplica nada. Cada lance carrega um identificador único gerado no celular, e o servidor recusa identificador repetido.

R10. Um lance vira erro pela ausência de impacto dentro da janela, nunca por aviso do dispositivo. O sensor não tem como reportar o que não aconteceu.

4. Pré e pós-condições

Para cada operação principal, o que precisa ser verdade antes e o que fica garantido depois. As pré-condições são o estado de onde a seta sai. As pós-condições, o estado onde ela chega mais o efeito que ficou registrado.

Operação	Pré-condições (antes)	Pós-condições (depois)
Parear com o alvo (c)	O alvo está ligado e se anunciando, o aplicativo tem permissão de Bluetooth, e o estado é S0.	Existe uma conexão ativa, o aplicativo já sabe o identificador, a bateria e o estado do boneco, e o estado é S1. Se a conexão falhar, continua em S0 e aparece a tela 2.4.
Calibrar (k … f)	O estado é S1 ou S3. Recalibrar um alvo já pronto é permitido, e a conexão está ativa.	Existe uma referência de força gravada, igual à mediana das cinco batidas, e ela sobrevive ao desligamento do boneco. O estado é S3. Uma calibração abandonada antes da quinta batida não grava nada, e a referência antiga continua valendo.
Iniciar sessão (s)	O estado é S3, existe um treino selecionado com versão publicada, e a quantidade de lances e o intervalo estão definidos.	O boneco recebeu o TREINO:n, a sessão tem um identificador próprio gerado no celular, o cronômetro está correndo e o estado é S4. Nenhum ponto foi contado ainda.
Registrar lance (h ou m)	O estado é S4 e ainda faltam lances para completar o treino.	A sessão tem um lance a mais, com identificador único, instante, tipo e classificação. A sequência de acertos aumentou (no h) ou voltou a zero (no m), e o estado continua S4. Se esse era o último lance previsto, a sessão encerra sozinha.
Pausar (p)	O estado é S4.	Nenhum impacto é registrado até a retomada, mas a pontuação, a sequência e os lances já contados continuam intactos. O estado é S5.
Retomar (r)	O estado é S5.	A contagem volta do ponto em que parou, sem reiniciar a pontuação. O estado é S4.
Encerrar (e)	O estado é S4 ou S5.	A sessão tem hora de encerramento, o resumo está calculado (pontuação, precisão, melhor sequência, acertos, erros e KOs), e ela está marcada como pendente de sincronização. Nenhum evento posterior altera nada disso. O estado é S6.
Sincronizar (fora do autômato da sessão)	A sessão está encerrada, há internet e o usuário está autenticado.	A sessão e os lances dela existem no servidor exatamente uma vez, mesmo que o envio tenha acontecido duas. A pontuação que aparece na tela passa a ser a recalculada pelo servidor, e a sessão deixa de estar pendente.

5. Modelo computacional

O **autômato finito determinístico** continua dando conta do controle da sessão, pelo mesmo motivo da Entrega 1. Mas implementar o sistema revelou um ponto em que a coisa sai da classe regular, e esse ponto nós deixamos de fora do autômato de propósito.

Por que o autômato finito basta

Quando desenhamos a máquina, a pergunta foi a mesma de antes: o que o sistema precisa lembrar para decidir se um comando pode ser aceito agora? As respostas continuam sendo poucas: pareado ou não, calibrado ou não, rodando, pausado ou encerrado. Nenhuma decisão de controle depende de quantos lances já aconteceram.

O sistema tem contadores, mas todos eles são **limitados**, e é isso que mantém tudo dentro da classe:

Contador	Até quanto vai	O que isso custaria
Batidas de calibração	0 a 5	Seis estados
Lances da sessão	0 a 200	Caberia nos estados, ao preço de multiplicá-los por 200
Melhor sequência	0 ao total de lances	Mesma coisa

Um contador que para num teto conhecido sempre cabe num autômato finito: é só ter um estado por valor. O que cresce é o tamanho do desenho, não a classe do modelo. Por isso nós tratamos a quantidade de lances como pontuação, calculada sobre os eventos, e não como parte do controle: o autômato fica legível e o resultado é exatamente o mesmo.

Se o treino não tivesse fim, encerrado só quando o jogador quisesse, aí o contador precisaria de memória sem limite e o modelo certo passaria a ser um autômato com pilha. Como na Entrega 1, foi a regra do jogo que definiu a classe do modelo.

Onde o sistema sai da classe regular

A parte nova é a sincronização. Ela é idempotente: mandar o mesmo lote duas vezes não duplica lance nenhum. Para isso funcionar, o servidor precisa responder a uma pergunta: esse identificador já chegou antes?

E aqui está o detalhe interessante: sobre um alfabeto ilimitado de identificadores, **a linguagem dos lotes sem repetição não é regular**. Um autômato finito tem um número fixo de estados, e pelo lema do bombeamento ele não consegue distinguir uma quantidade arbitrária de identificadores diferentes. Passado certo ponto, dois identificadores distintos caem no mesmo estado, e a repetição passa batida.

O que o sistema usa no lugar é memória externa sem limite: o índice único sobre a coluna de identificadores, dentro do banco de dados. Em termos de modelo, isso corresponde a uma máquina com memória de trabalho irrestrita, não a um autômato finito.

A conclusão

Não existe um modelo só para o sistema inteiro. O **controle da sessão** é um autômato finito determinístico, e é ele que este documento modela. A **persistência idempotente** precisa de mais poder de computação e fica fora do autômato.

Isso não é defeito do modelo, é o reconhecimento de que são duas responsabilidades com necessidades diferentes. E é justamente o que permite ao firmware rodar a sessão com um `switch` de poucos estados, enquanto a deduplicação acontece noutra camada, com outro tipo de memória.

6. Análise de decidibilidade

Todas as decisões que o sistema toma são decidíveis: existe um procedimento que sempre termina e sempre responde sim ou não. Nenhuma delas depende de prever o comportamento de um programa qualquer, que é onde mora a indecidibilidade.

Decisão	Como é tomada	Por que sempre termina
Esse comando vale agora?	Consulta à tabela de transições pelo par (estado, símbolo)	A tabela é finita e a consulta é uma leitura
Dá para concluir a calibração?	Compara o contador com 5	É comparar dois inteiros
A sessão acabou?	Compara os lances registrados com os configurados	Idem

Decisão	Como é tomada	Por que sempre termina
Esse impacto foi KO?	Compara o ângulo de pico com o limiar de 65°	Comparação de dois números
Esse token é válido?	Confere a assinatura e a data de expiração	Hash de tamanho fixo e comparação de datas
Essa pessoa pode ver esse treino?	Consulta ao banco por escopo e dono	Consulta sobre um conjunto finito de linhas
Esse lance já chegou?	Busca o identificador no índice único	Busca numa estrutura finita a cada instante

Por que elas são determinísticas

Uma decisão é determinística quando, para a mesma entrada, o resultado é sempre o mesmo e não há escolha entre caminhos.

No autômato da sessão isso é estrutural, não é promessa: para cada par de estado e símbolo existe **exatamente um** destino, contando o estado de erro para todos os pares que não aparecem no desenho. Não há transição vazia nem ambiguidade. É por isso que a tabela de transições vira uma função só dentro do firmware, consultada no laço principal.

Nas decisões sobre dados, o determinismo vem do mesmo lugar: comparar números, conferir uma assinatura, consultar um conjunto finito. Nenhuma delas envolve busca sem limite, recursão sem base ou dependência do relógio.

Um limite que vale deixar registrado

A pergunta “esse identificador já chegou antes?” é decidível, mas não por um autômato finito, como a seção 5 mostrou. São perguntas diferentes: uma decisão pode terminar sempre e ainda assim precisar de mais memória do que um autômato finito tem.

Na prática ela termina porque o conjunto de identificadores guardados é finito a cada instante, mesmo que cresça sem teto teórico. A busca percorre um índice que existe, não um espaço infinito.

7. Divergências em relação à Entrega 1

São seis diferenças entre o que nós modelamos na Entrega 1 e o sistema que acabou sendo construído. Nenhuma é correção de erro: todas vieram de decisões tomadas durante o desenvolvimento.

#	Entrega 1	Implementado	Por que mudou
1	k calibrar era um evento instantâneo	Virou uma sub-máquina de cinco batidas, com k abrindo e f fechando	A tela precisa mostrar o progresso batida a batida, e a referência é a mediana das cinco
2	Depois de conectado, não se voltava a S0	A desconexão pode acontecer em qualquer estado conectado	O Bluetooth cai na prática; ignorar isso deixava o modelo incompleto
3	m era gerado pelo dispositivo	Passou a ser gerado pelo aplicativo	O sensor inercial não detecta ausência de impacto
4	Feedback com áudio e LED	Só áudio	A fita de LED saiu do escopo do MVP
5	A classificação do impacto vinha da força	Vem do ângulo de inclinação, em quatro faixas	Medir inclinação resiste melhor à posição em que o sensor foi montado, e é o que permite detectar o KO

#	Entrega 1	Implementado	Por que mudou
6	O final era a sessão encerrada	Depois do encerramento vem a sincronização idempotente	O aplicativo funciona offline, e reenviar precisa ser seguro

A garantia da Entrega 1 continua valendo

Na Entrega 1 nós provamos, nas três representações, que nenhuma batida podia ser pontuada antes da calibração: no autômato, só se chegava no estado de execução passando por conectado e calibrado. Na expressão regular, nenhum `h` aparecia antes do bloco inicial. Na gramática, a variável que produz `h` só era alcançada depois de `c`, `k` e `s`.

Essa propriedade sobreviveu à implementação, e o diagrama revisto continua mostrando isso: **toda seta que chega em S4 vem de S3, e a única entrada de S3 é por f, que por sua vez só sai da posição 5/5 da calibração.**

Uma simplificação que nós mantivemos

Na Entrega 1 assumimos que os valores numéricos do impacto viajam junto com o evento mas não entram no alfabeto, senão ele ficaria infinito. A mesma decisão vale agora para as quatro faixas de classificação. São só quatro valores, poderiam entrar, mas não mudam nenhuma transição, e colocá-las ali multiplicaria os símbolos sem alterar comportamento nenhum.